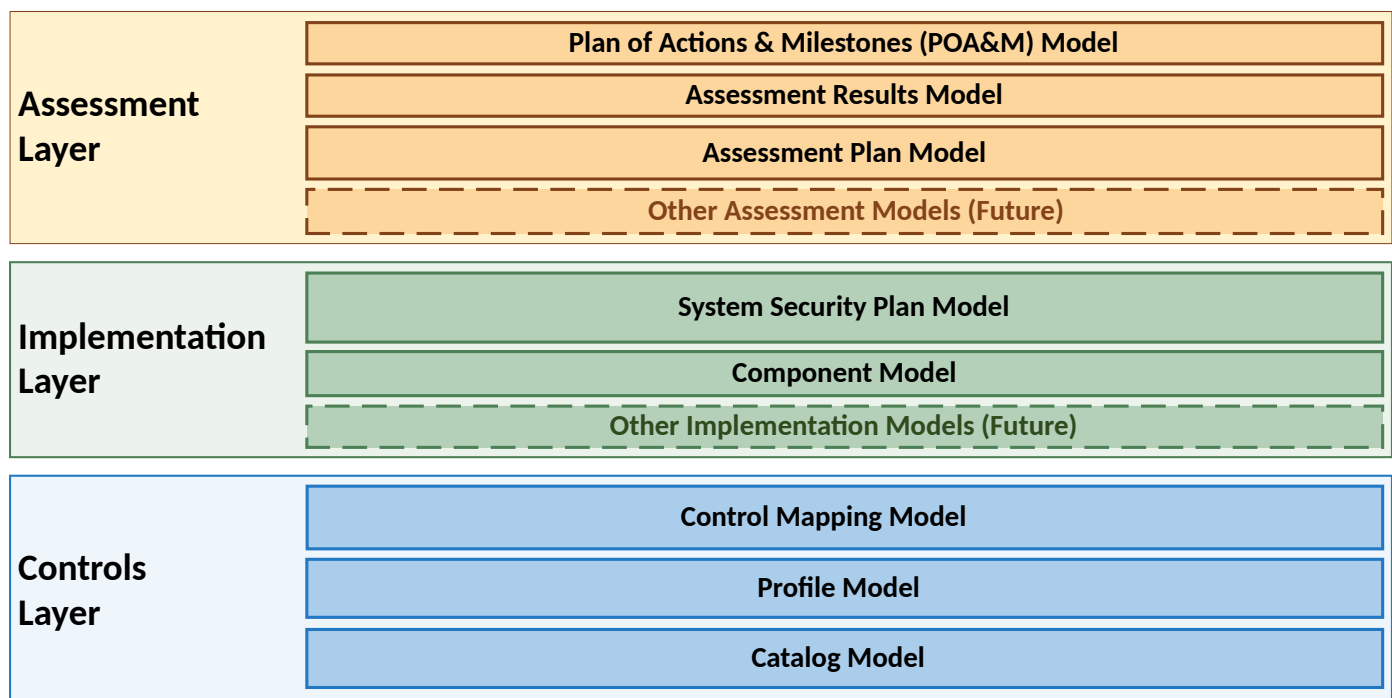


OSCAL

Layers and Models

The OSCAL architecture is organized in a stack of *layers*. Each lower layer in the stack provides information structures that are referenced and used by each higher layer. Each layer is composed of one or more *models*, which represent a information structure supporting a specific operational purpose. Each model in OSCAL is intended to build on the information provided by the model(s) in the lower layers.

The following image depicts each layer and the corresponding model(s) for each layer.



Each OSCAL model is represented in multiple, machine-readable *formats* (e.g., XML, JSON, YAML), which provide a serialization and encoding mechanism for representing and exchanging OSCAL data, also referred to as *OSCAL content*.

Individual layers are summarized below, with links to additional layer and model information. There is also an [introduction to the OSCAL models](#), as well as information related to the [terminology](#) used in describing each model.

The OSCAL Layers

The diagram above identifies the OSCAL layers from the bottom up because each higher layer relies on the layer beneath it. The following OSCAL layer discussion begins with the bottom layer in the diagram, addressing each in order moving toward the top of the diagram.

The OSCAL layers are:

- [The OSCAL Layers](#)
 - [Control Layer Overview](#)
 - [Implementation Layer Overview](#)
 - [Assessment Layer Overview](#)
- [Status of the OSCAL Layers](#)
- [Well-formed Data and Valid OSCAL](#)

Control Layer Overview

Cybersecurity frameworks often define a set of **controls** intended to reduce the risk to a system.

Framework authors typically organize these controls into a **catalog**.

Organizations and system owners identify which controls are applicable to a system, which may include controls from more than one catalog. These are often referred to as a **baseline** or **overlay**.

The OSCAL [Control Layer](#) provides:

- The **Catalog model** for expressing and organizing these controls in a standard, machine-readable format. Controls from any control-based framework can be expressed in an OSCAL catalog.

The catalog model is the basis for all other OSCAL models. Controls used in any other OSCAL model must first be *defined* in this model.

Controls may include statements (requirement definitions), parameters, references, objectives, and assessment methods. The model also enables the controls to be organized.

- The **Profile model** for selecting, organizing, and tailoring a specific set of controls.

A profile enables controls to be selected and tailored to express a baseline of controls. A control used in the implementation, assessment, and assessment results layers must first be imported by a profile.

After importing a control, a profile may be used to tailor the control. This includes additions, changes, and removal of statements, parameters, control objective, and assessment actions.

OSCAL profiles can import controls from catalogs or other OSCAL profiles, enabling the creation of a new baseline by customizing an existing baseline. This approach provides full transparency and traceability of control tailoring from a baseline back to the original catalog's control definition. Every control imported by a profile must originate within a catalog.

Organizations already perform such selections manually. OSCAL profiles enable automation of control selection and tailoring while providing traceability.

- The [Control Mapping Model](#) is for representing relationships among controls and control elements, describing how controls and requirements relate to one another across authoritative sources.

A mapping model is a structured way to link different security control frameworks, standards, or baselines to each other. The mapping model identifies the source and target frameworks. The source framework is the one being mapped from, and the target framework is the one that is being mapped to.

Each control in the source framework is linked to one or more or no corresponding controls in the target framework. Each mapping can include metadata, such as the type of relationship (e.g., "equivalent-to", "subset-of", "intersects-with", "no-relationship"), and references.

OSCAL's Mapping model reduces duplication of effort when complying with multiple frameworks, enables automated analysis of control coverage across standards, and provides clarity on how different frameworks relate to each other, helping organizations streamline compliance and assessment processes.

Implementation Layer Overview

The OSCAL [Implementation Layer](#) focuses on the implementation of a system under a specific baseline as well as the individual components that may be incorporated into a system. A component is anything that can satisfy a control, such as a policy, process, compliance artifact (such as FIPS 140-2 validation), as well as hardware, software, and services.

The OSCAL [Implementation Layer](#) provides:

- The [System Security Plan \(SSP\) model](#) enables a system owner to express the security implementation of an information system within the context of a specific baseline (OSCAL profile).

SSPs expressed in a machine-readable format can be easily imported into a tool, allowing for increased automation of SSP validation and authorization.

An OSCAL SSP can also be transformed from the machine-readable form to a human-readable version.
- The [Component Definition model](#) is intended to define information about an individual *component*, such that its contents can be imported into an OSCAL SSP.

The model enables an organization or component creator to provide a description of the component and applicable security configuration information. It can also describe how a specific configuration satisfies the controls of an identified baseline. SSP authoring tools will be able to pre-populate significant portions of an SSP by importing this content and allowing it to be tailored to reflect the actual implementation within the system.

Assessment Layer Overview

The OSCAL [Assessment Layer](#) focuses on assessment activities, on communicating all assessment findings including supporting evidence, and identifying and managing the remediation of identified risks to a system identified as a result of assessment activities. This includes the ability to report all findings.

The assessment layer will be expanded in OSCAL 2.0 to support the automation of assessment activities.

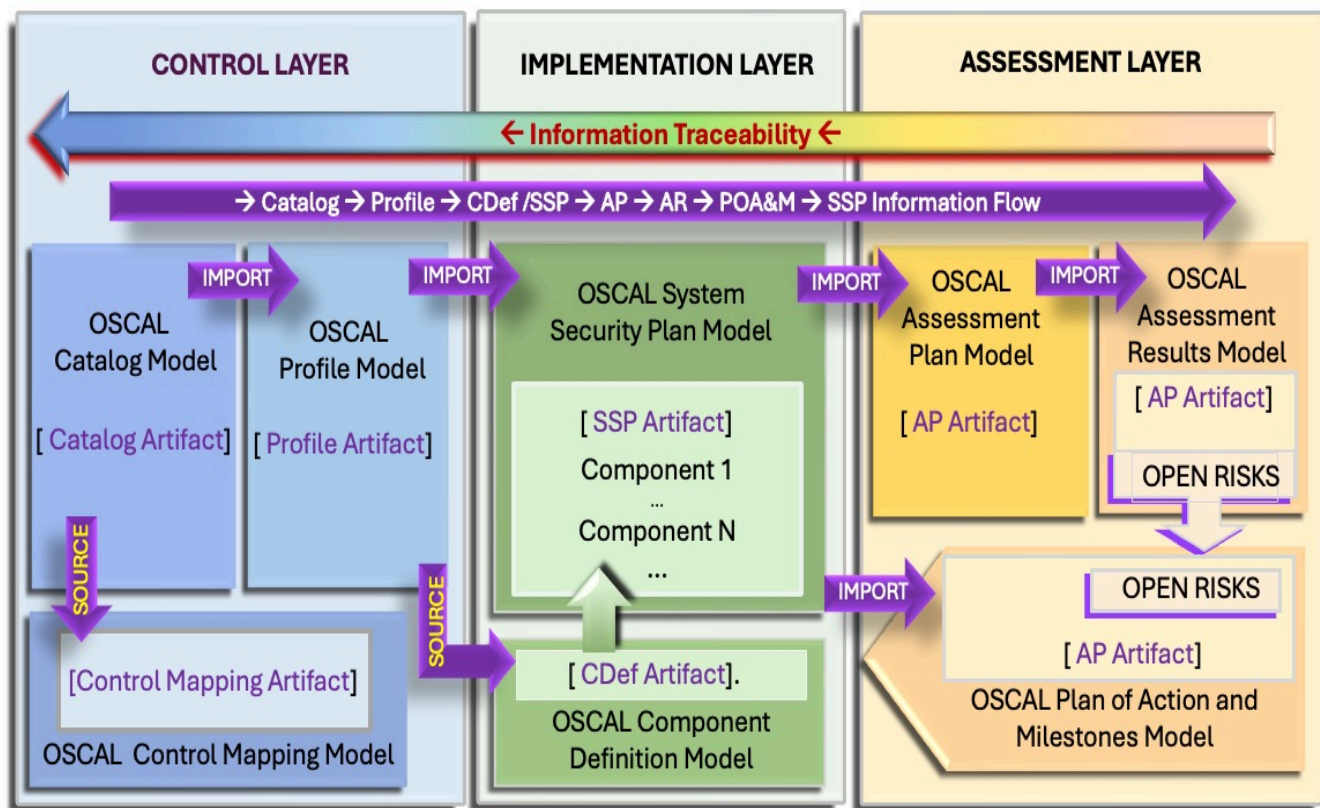
Under OSCAL, assessments are always expressed in the context of a specific system implementation relative to a defined set of controls. OSCAL supports both continuous assessment as well as traditional "snapshot in time" assessments.

The OSCAL [Assessment Layer](#) provides:

- The [Assessment Plan model](#) allows assessment plan information to be described, including how and when a system assessment is intended to be performed, the scope of the assessment, and what assessment activities should be conducted.
- The [Assessment Results model](#), which represents information produced from a set of assessment activities, to include when the assessment was performed, the assessment scope, evidence collected during an assessment, and any assessment findings. The assessment model supports information from periodic and continuous assessments.
- The [Plan of Action and Milestones \(POA&M\) model](#), which represents a set of findings for a periodic or continuous assessment that need to be addressed by the system owner/maintainers.

Information Traceability and Data Flow

The OSCAL models provide native traceability from the assessment layer to the catalog layer. Each OSCAL model defined an import mechanism for the OSCAL artifact it uses. The diagram below depicts the native traceability and the direction of the information flow supported by all OSCAL models.



Status of the OSCAL Layers

The OSCAL layers described above provide a framework for organizing the OSCAL models. As the OSCAL project [progresses](#), the features of these models are expected to evolve and expand; the layer descriptions above are included here to indicate the current status of the related models within OSCAL and may not represent the final features supported by each layer and their corresponding model(s). XML, JSON, and YAML formats for each model will be provided when the model is released.

The new OSCAL models can be in various states of readiness for release:

- **Future:** Additional models will be published as prototypes in the future.
- **Early Access Draft:** Indicates that the model has been newly developed and released for review and comment, but has received little to no review. Multiple revisions will likely be needed to mature the model for release.
- **Draft:** Identifies that the model has received some review, but not enough review to be released as a final version. A revision or two might be needed to prepare the model for release.
- **Prototype:** The model has been internally reviewed and considered fairly stable. The model is made available for public review.
- **Pre-Final:** The model is fairly stable and has been extensively reviewed. Some small changes might occur before final release, but the impact of these changes will be small to users or tool

implementers.

- **Released:** The model has been released. Only non-backwards compatibility breaking changes will be made in maintenance releases.

The following is the release state of each model, along with download links for the latest versions of XML and JSON schema for each model. YAML is also supported through conversion between JSON and YAML. Since YAML is a superset of JSON, some YAML tooling allows JSON schema to be used for YAML validation. In this way, the provided JSON schema supports both JSON and YAML.

Layer	Model	Current State	Reference	Schemas
Control	Catalog	Released	XML, JSON, YAML	XML, JSON/YAML
Control	Profile	Released	XML, JSON, YAML	XML, JSON/YAML
Control	Control Mapping	Released	XML, JSON, YAML	XML, JSON/YAML
Implementation	Component Definition	Released	XML, JSON, YAML	XML, JSON/YAML
Implementation	System Security Plan	Released	XML, JSON, YAML	XML, JSON/YAML
Assessment	Assessment Plan	Released	XML, JSON, YAML	XML, JSON/YAML
Assessment	Assessment Results	Released	XML, JSON, YAML	XML, JSON/YAML
Assessment	Plan of Action and Milestones	Released	XML, JSON, YAML	XML, JSON/YAML

Well-formed Data and Valid OSCAL

[Per our guidance](#), OSCAL-enabled tools must check OSCAL document instances to ensure they are well-formed and valid based upon the models in these layers.

This page was last updated on January 29, 2026.